The State of Test Automation in DevOps: A Systematic Literature Review

Akshit Raj Patel
CSE & IT Jaypee Institute of Information Technology, Noida, India
aksh.patel78@gmail.com

Sulabh Tyagi
CSE & IT Jaypee Institute of Information Technology, Noida, India
sulabhtyagi2k@yahoo.co.in

ABSTRACT

DevOps is becoming increasingly popular in the software industry to deliver quality software in less time. However, building a DevOps pipeline is not easy. Many factors need to be considered in the process, and many decisions need to be made. Making the DevOps lifecycle successful, "Test Automation" played a pivot role in different stages of the DevOps pipeline. With automated testing, continuous testing can be achieved - a crucial driver for delivering high-quality software rapidly and a vital factor in reducing risks, increasing productivity, and lowering costs. Through automated testing tools, the entire process of testing can be automated. This paper presents a review of the use of Test Automation in a DevOps environment, it also provides glimpses of automated testing tools. The results are limited to articles published in peer-reviewed conferences, scientific journals, and books published between 2013 and 2021. The articles have been synthesized by categorizing the articles based on the research method, benefits of automated testing in DevOps, and the tools used for performing testing in an automated form. Our results shed light on different studies related to testing automation as one of the DevOps practices and its impact on software quality.

CCS CONCEPTS

• **DevOps and Automated Testing**; • **Testing Tools**; • **Systematic Literature Review**;

KEYWORDS

Test Automation, DevOps, Testing, Quality Assurance, Test Data, Test Strategies

ACM Reference Format:
Akshit Raj Patel and Sulabh Tyagi. 2022. The State of Test Automation in DevOps: A Systematic Literature Review. In *2022 Fourteenth International Conference on Contemporary Computing (IC3) (IC3-2022), August 04–06, 2022, Noida, India.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3549206.3549321

1 INTRODUCTION

The adoption of agile practices began in the early 2000s when companies adopted an accelerated development lifecycle that involved frequent customer feedback. The adoption of tools enabling continuous integration and delivery, which automate the build, test, configure, and deploy processes, lead to the development of continuous integration and delivery tools.

As a result, critical functions like testing, development, and delivery to production were managed by separate teams that operated in their silos. It created inefficiencies and slowed the software development process. It also brought about DevOps, the organizational philosophy, practices, and tools that enable small, cross-functional teams to deliver and maintain the quality of end-to-end product updates continuously.

In the early days of DevOps, it unified just development and IT operations, with testing still mainly performed manually by a separate team. This made it easier to deliver and monitor cloud-based applications. The process also resulted in fully automated CI/CD pipelines. However, it did not result in much faster release cycles since testing was often siloed and required manual intervention. Rather than relying on centralized QA teams, organizations are now embedding QA within the entire development team to alleviate the testing bottleneck. DevOps emphasizes automating all software development processes to increase speed and agility parameters. It includes automating the ***testing process*** and ***configuring it to run automatically***. For this, DevOps must build a mature automation testing framework that will assist in scripting test cases. Fortunately, many tools make test automation easy. For automated testing, it is essential to select the right tools. Testing software applications through automation can enhance efficiency and effectiveness, but working with the wrong tools can negatively impact the overall process. Some of the tools available on the market include ***Jenkins*** *(open-source DevOps testing tool),* ***Bamboo*** *(Continuity Integration tool, also used for DevOps testing),* ***Apache JMeter*** *(open-source load testing tool),* ***Selenium*** *(one of the most popular automated testing tools),* etc. Automated testing requires skills, not just tools. Testing automation without the necessary skills can pose a serious risk to many DevOps projects. A test automation engineer must know several areas, such as the underlying technical landscape, automation tools, and how to write scripts simultaneously with development. Automated tests require collaboration with both development and operations teams. To maximize test coverage, the collaboration will be vital to developing test scripts. The user might not get the value provided by focusing on one tool. By performing a lightweight review of literature related to test automation as a DevOps practice, this study examines test automation's role and its impact on the DevOps-based software development process. This paper is divided into seven sections. Section 1 provides a brief overview of test automation, automated tools, and the relation between DevOps and test automation. Section 2 gives the background to stages in testing practices, Automated testing in DevOps, and revamping the role

IC3-2022, August 04–06, 2022, Noida, India

ACM ISBN 978-1-4503-9675-2/22/08…$15.00
https://doi.org/10.1145/3549206.3549321

of QA while applying automated testing. Section 3 discusses the review methodology of the present study. Results and Discussion part are discussed in Section 4, and the paper's conclusion is in Section 5.

2 BACKGROUND

This section provides a brief overview of stages in testing practices, DevOps testing automation, and Changement in the role of QA while applying automated testing.

2.1 Stages in Testing Practices

The practice of test automation is to automatically review and validate a software product, such as a web application, to ensure it meets predefined quality standards regarding code style, functionality (business logic), and user experience. Typically, testing involves the following stages:

- A unit test validates a unit of code, such as a function, to work as it should.
- An integration test ensures that several pieces of code will work smoothly together without causing unintended effects.
- An end-to-end test is a way to verify that the application fulfills the user's expectations.
- During exploratory testing, numerous parts of the application are examined from the user's perspective to determine if there are any functional or visual problems.

The different types of testing are visualized as a pyramid. With each step up the pyramid, the number of tests decreases, and the cost of creating and running them increases. All testing within the pyramid was done manually in the past. It was a time-consuming, costly, and error-prone task until automated testing tools were developed. Almost all unit tests are now fully automated, and automating unit tests is considered a best practice. In addition, integration tests are often automated, or, if they are not, they are usually skipped in favor of end-to-end tests. Much of the effort is devoted to automating the end-to-end layer of the testing pyramid, thus reducing the need for integration tests. As part of the DevOps cycle, QA teams must ensure that testing is automated and near 100% code coverage is achieved. There needs to be standardization of environments and deployment automation on their QA boxes. The continuous integration cycle should include pre-testing, cleanups, post-testing, etc. Automating these activities will ensure they are streamlined. With low-code tools like *mabl*, team can automate all CI/CD pipeline stages to ensure reliable and consistent end-to-end testing. This allows catching issues much earlier in the development phase. The earlier team uncover problems with a release, the easier and less expensive it is to rectify them.

2.2 DevOps Automated Testing

In practice, this implies that developers prefer to write unit tests to ensure that their code works as intended. In contrast, quality practitioners and product owners prefer to design automated UI tests to ensure that the end-to-end user experience is validated. Exploratory testing sessions are also organized by quality practitioners, in which the team manually investigates various application areas for faults. Running automated tests as early and as frequently as feasible within the CI/CD pipeline is a DevOps best practice. It proactively monitors user experience concerns by executing automated UI tests in production. Because today's apps rely on a large number of services with many moving components, synthetic transaction monitoring by conducting tests in production might uncover issues with third-party services before your users do. Testing automation isn't a one-size-fits-all strategy, but a few factors are to consider:

- **Frequently Release:** With more frequent releases, teams need to devote more time to testing automation, especially end-to-end testing that should be performed on every release. As a first step in accelerating their release cycle, the team could perform more unit tests and create simple automated UI smoke tests to perform a quick sanity check on every build.
- **Tools Available:** With modern test automation tools, the development team will be significantly more capable of consistently delivering high-quality software. Evaluation of testing tools should consider the ease of creation of tests, their reliability, their maintenance requirements, and their integration with your CI/CD pipeline.
- **Level of maturity of the product:** If the development team works on a product with numerous existing customers and a mature codebase, a team already has an established release cadence and testing practices. As the team moves to continuous integration or entire CI/CD, it's essential to include test automation as a crucial part of pipeline automation. Fast delivery and feedback are unsustainable without automating testing earlier and throughout development. Focus on defining end-to-end test cases for each feature from the start and set a target for unit testing coverage. Before adding automated end-to-end tests to an element, it's best to wait until it's close to a release so the team can avoid test failures from breaking UI changes.
- **Data testing and CI/CD infrastructures:** Test automation is in itself a challenge. Despite this, teams have difficulty adopting test automation earlier in the pipeline due to the lack of pristine test environments. Therefore, having a team discussion early in the development process about testing strategy and creating the necessary testing infrastructure is crucial.

2.3 Role of QA in automated testing in DevOps

QA was traditionally responsible for writing test cases, conducting manual tests, and reporting issues. In the product group, automation engineers accounted for a relatively small proportion of the quality professionals, mostly manual testers. In addition, test automation engineers were required to have a strong background in technical skills, some development skills, and a solid understanding of business requirements. This particular skill set used to be in high demand and short supply. As a result, quality assurance was heavily reliant on manual testing.

With multiple releases to production every day, DevOps changes everything. Testing a build should only take a few minutes, not days. As advocates for the user, teachers of best practices, and help achieve the benefits of end-to-end testing, software teams need

qualified professionals to lead and coach the rest of the team - especially developers. Test automation tools like *Mabl* can help quality analysts and manual testers become automation engineers thanks to low-code tools. The less time QAs spend manual testing, the more time they can devote to playing a strategic quality coaching role and assisting all team members in the quality assurance process.

3 REVIEW METHOD

As part of this review, a review protocol was created, followed by developing an inclusion and exclusion criterion, a search for relevant studies, and the extraction and synthesis of data. These stages and methods will be described in detail in the rest of this section.

3.1 Research Questions

The following research questions were set up when we decided to conduct a systematic review.

RQ1: What are the trends in Test Automation practices in DevOps?

RQ2: What are the benefits of Test Automation in DevOps?

RQ3: *Which testing tools are available for test automation in a DevOps environment, and at what stage are they used?*

3.2 Review Protocol Development

Using Kitchenham's [6] guidelines, our review protocol was developed to include research questions, search strategies, inclusion and exclusion criteria, data extraction, and synthesis.

3.3 Data Source and Search Strategy

In search of relevant papers in our current research area, we used the Google Scholar database. Among the papers we found, some are from the following electronic databases also:

- IEEE Xplore
- Springer Link

Our search included not only the databases mentioned above but also all the volumes of the following conference proceedings:

- ACM
- XP

Figure 1, illustrates our systematic literature review process and the number of studies identified at each stage. The titles, abstracts, and keywords of articles in the database and conference proceedings included in stage 1 were searched by using the following search terms:

- Test Automation AND DevOps
- Testing AND DevOps
- Quality Assurance AND DevOps
- Test Strategy AND DevOps
- Test Data AND DevOps
- Automated Testing AND DevOps

To return the research article, any of the above terms had to appear in the search terms. The Boolean "OR" operator combined the above search terms. As a result, we searched using the following string combination, "***a OR b OR c OR d OR e OR f".*** We found 1169 articles using the above search string.

Table 1: Quality Assessment Questionnaires Table

	QUESTIONAIRE	TITLE
1.	Do the questions prepared for research (RQs) in the current review appear to be well defined?	Research
2.	During the research process, are the "Objectives and Aims" clearly defined or not?	Aim
3.	How does the study describe the various stages of testing with its impact on quality assurance in DevOps? How are testing tools utilized to automate the process, and at what stage have they been adopted?	Context
4.	Has the research methodology been clearly defined?	Research Design
5.	Does the study discuss how test automation can benefit DevOps?	Value

3.4 Inclusion and Exclusion Criteria

During stage 1, we fetched a total 1169 articles from the selected databases based on our search strategy. During stage 2, we examined the titles of all 1169 articles from stage 1 to determine their relevance to the current systematic review. At this point, 848 articles were excluded from our study, along with studies that were not related to testing automation. Articles with titles that were not in the scope of our study were also excluded. Our focus area was test automation in DevOps, so the abstracts of the remaining 321 articles were screened for relevance at stage 3. As a result of reviewing the abstracts of these articles thoroughly, we select only relevant articles that meet the objective of the current systematic review, the result of stage 3 is 48 articles. A total of 48 articles were taken for analysis at stage 4, and the criteria for analysis were *rigor, relevance, and credibility* as outlined in the Critical Appraisal Skills Programme (CASP) [7] [8]. A total of 28 articles were excluded after a series of reviews, with the remaining 20 articles selected for data extraction.

3.5 Analyzing and Synthesizing Data

We pulled the data from 20 primary plus eight secondary studies and compiled them in MS Excel. The items of each excel sheet form were planned according to our objectives and to answer our research questions. To extract the required data, we studied the full text of each paper in detail, including test automation in DevOps, testing stages, automating testing's impact on quality assurance, and the research methodology. By studying each article closely, we classified the paper's contents according to the research method employed, the publication year, the study context, and the findings.

3.5.1 Quality Assessment. The following quality criteria were developed to check the content quality present in the studies. Identifying good primary studies involves a thorough quality assessment. There are three criteria regarding the quality of the description of a study's rationale, aims, and context. So, each study was evaluated under the following criteria: Rigor, Credibility, and Relevance. Accordingly, a CASP'S based criteria Quality Assessment questionnaire was prepared, as shown in Table 1

Only those papers that secure a score of 3 or higher will be considered for further study, while those below the 3-point threshold will not be considered.

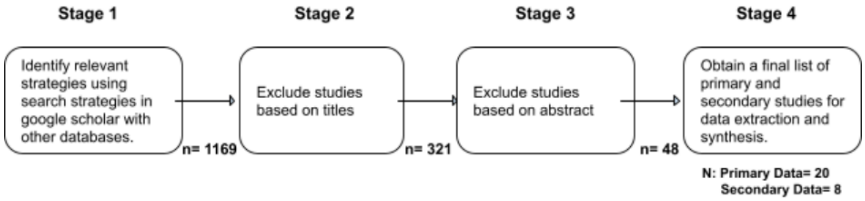

Figure 1: Selection of studies at various stages

Table 2: Quality Assessment Count Table

Articles	Q1	Q2	Q3	Q4	Q5	Total
	RESEARCH	AIM	CONTEXT	RESEARCH DESIGN	VALUE	
[2]	1	1	1	1	0	4
[3]	1	1	1	0	0	3
[9]	1	1	1	1	1	5
[10]	1	1	1	0	1	4
[11]	1	1	0	1	1	4
[35]	0	1	0	1	0	2
[36]	0	1	0	1	0	2

On a thorough selection of 48 research papers from various data sources, the Systematic Literature Review (SLR) was conducted, and analysis was done based on quality assessment questionnaire parameters. An article with a count of 3 or more was selected, and a total of 20 primary studies were selected, which gives a proper answer to the current study's research questions.

4 RESULTS AND DISCUSSION

A total of 28 studies discussed DevOps, automating test cases, and the impact of test automation tools on DevOps-based development in one way or another. These 20 studies are primary studies [2] [3] [9]-[26] which come after qualifying the quality assessment phase, and eight are secondary studies i.e., [27] [28]-[34]. The following section delves into primary studies classified using different research methods, databases, and publication years.

4.1 Study Types and Classifications

In Figure 3, primary studies are classified based on the type of research methods used. Based on the research methods used, Figure 3 shows the number of publications.

4.1.1 Single-Case Study and Multi-Case Study. Using FP7 HARNESS as a case study, Stillwell and Coutinho [10] describe the development and deployment infrastructure created to support the integration efforts of the project. As a result, authors have used automated testing and deployment in their work, which has significantly impacted the pace and efficiency of their work. This study [17] is also significant Akman et al. discusses how they setting up and using the tool infrastructure to manage traceability between different work items such as change requests, configuration items, technical documents, test cases, technical tasks, and code; to utilize the knowledge base and history during impact analysis; to monitor real-time the status of releases, change requests, sprints, test plans, and all the related work items; to generate packaged documentation for customers during long-term maintenance. Kalliosaari et al. [11] showed a qualitative work as authors taken three software development organizations in Finland were interviewed for their qualitative multiple-case study. According to the responses, practitioners can increase release frequency with DevOps and improve test automation practices with DevOps. The concept of DevOps was seen to boost communication and employee welfare among departments by encouraging collaboration.

4.1.2 Experience Report. An experience report on TAPI is an F-Secure DevOps team presented by Wang et al. [22] As part of F-Secure's TAPI culture, the team develops software for Windows

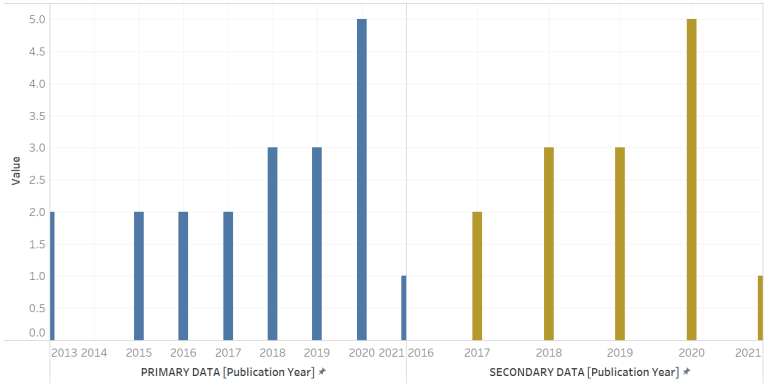

Figure 2: Data collection count with respect to Publication Year.

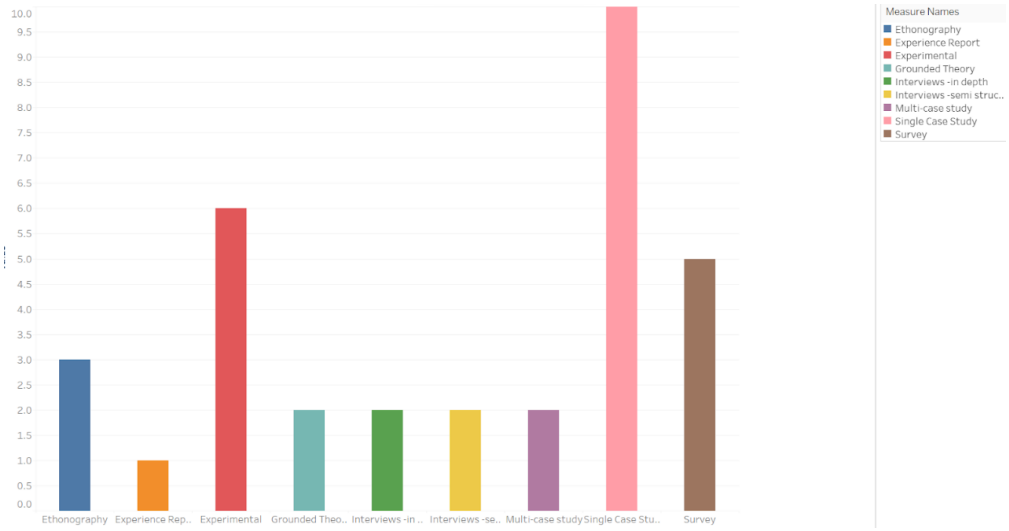

Figure 3: Primary Data count with respect to Research Methodology.

applications. The team highly rates test automation for continuous development for its maturity and satisfaction. Authors examined experience notes, reflection reports, and telemetry results to study their TAPI. According to the authors, test automation maturity for continuous development can be defined as a set of indicators, such as the increasing speed of releases, the improvement of the team's productivity, and high-test efficiency.

4.1.3 Grounded Theory. An organization's proficiency in delivering services and applications at high velocity requires competing effectively in the market. Such management processes' practices and tools demand a quick and reliable model. When building cloud applications, it is necessary to start automating processes at the software engineering level, while DevOps processes use cloud and non-cloud DevOps automation tools. Agrawal and Rawat [20], through their work, tried to show a shift from DevOps to Cloud for achieving more agileness at software development and operations. In parallel, consideration of extending those DevOps processes and automation into public or private clouds is the prime approach for the author's work.

4.1.4 Experimental. Borgenholt et al. [2] work demonstrates an approach to automated testing and quality assurance in cloud environments, considering deployment cost. With a distributed service architecture and some given performance goals, the result will suggest the optimal resource type and filesystem with the lowest price point for each function of the architecture. The author's work provides a solution that is modeled after the auditioning process in the theater industry. It provides an approach that fits well into the author's work context and is easy to understand and follow. Audition's resulting tool is a working implementation of their model, and it is extendable in several ways, allowing for integration with local technologies.

4.2 DISCUSSION

The present literature review identified a significant number of studies that shed light on the automating the testing phase in DevOps and provides an answer to our research questions.

(RQ1). What are the trends in Test Automation Practice in DevOps?

Automation of testing has now been accepted as a DevOps best practice. It may seem daunting to automate a large portion of the development pipeline initially, but a single test can be automated and run regularly on a schedule to get started. As a result of automated testing, DevOps offers the following benefits:

- A faster development process without sacrificing quality
- A better way to collaborate with a team
- Test automation increases the coverage of releases, making them more reliable.
- By distributing development among several small teams that are self-sufficient, a team can produce consistent quality outcomes with reduced risk
- Security
- Improved customer satisfaction

Agrawal and Rawat [20], tried to show the trends in testing as they stated that adopting new paths for development and operations simultaneously seems problematic from the implementation end, so guidance is needed to handle such DevOps as a service to cloud applications. Unit, system, acceptance, and regression test automation are the primary automation expertise that may further promote continuous integration, continuous delivery, continuous deployment, and continuous monitoring. In testing, there has been a change in the roles and responsibilities of testers. Instead of testing being solely the tester's responsibility, there is a much more collaborative approach. Additionally, testers must work more closely with other stakeholders, including operations and business. Automated testing is a significant part of DevOps, but manual tests are still needed, which are more risk-based than in the past. The work of Cruzes et al. [19] provides insight into areas we should focus

Table 3: Automated Testing Tools applied in testing stages

Stages testing T..	Tool / Framework	Reference
Integration test..	Audition Frame..	[2]
Build Testing	Maven	[13]
CI/CD Stage	Jenkins	[9]
	SolanoCI	[24]
Continuous	Buildbot	[10]
Integration Stage	Jenkins	[13]
		[14]
		[16]
		[21]
ContinuousTesti..	SonarQube	[16]
Regression	Selenium	[13]
Testing		[15]
Smoke Testing	Rolling Upgrade ..	[12]
Unit Testing	JUnit	[13]

on to improve testing in the DevOps environment. Moreover, the author's work helps practitioners focus on specific areas that need improvement.

(RQ2). What are the benefits of Test Automation in DevOps?

To keep pace with DevOps, test automation is essential. DevOps helps achieve the benefits of moving faster and being more effective with test automation. Testing by hand has become a thing of the past. By doing so, the testing bottleneck is eliminated, releases are accelerated, and quality is improved. *Kalliosaari et al. [11]* used a qualitative multi-case study approach to interview representatives of three software development organizations in Finland. According to respondents, using DevOps can lead to more frequent releases and improved test automation practices. *According to the study, DevOps boosted communication and employee satisfaction by encouraging collaboration between departments. A continuous release pipeline allows for more experimentation and rapid feedback collection.*

(RQ3). Which testing tools are available for test automation in a DevOps environment, and at what stage are they used?

It is crucial to choose the right tools when automating tests. The most effective way to enhance the efficiency and effectiveness of software applications is automation testing, but using the wrong tools can negatively impact the overall process. In addition to Jenkins (open-source DevOps testing tool), Bamboo (Continuity Integration tool, also used for DevOps testing), Apache JMeter (a load testing tool), and Selenium (an automated testing tool), and some other tools are also available in the market. According to Table 3, there are articles that describe how researchers and practitioners have adopted testing tools and frameworks to automate different stages of testing in a DevOps environment.

Borgenholt et al. [2] proposed a framework titled "Audition." The approach used by Audition is in line with DevOps as it supplies a tool that can automate an otherwise cumbersome analysis process. This framework has many advantages, as a tool to track how performance demands increase across releases as a form of A/B testing system, as a tool to test configuration management policies, or lastly, as a simple smoke test/integration test that checks for a correct functioning of software before deployment.

Mitesh Soni [9] tried to create a proof of concept for designing a practical framework for continuous integration, testing, and delivery to automate the source code compilation, analysis, test execution, packaging, infrastructure provisioning deployment, and notifications using the build pipeline concept. The author used Jenkins to verify the integrated code with automated build and notify stakeholders of the status of build execution. Apart from this author also used Junit for doing automated testing/continuous testing by making Automated test cases, i.e., Unit test execution on integrated source code.

As part of the DevOps approach, Stillwell and Coutinho [10] used Ansible to manage the configuration and deployment of their testing environment. To enable automated testing, the authors used Buildbot in addition to python's implementation. Using Buildbot, it is possible to schedule various automated tests to run at specific intervals whenever a repository is changed. The architecture of Buildbot consists of one or more master nodes that monitor repositories and generate tasks and multiple build-slave nodes to run queued tasks. Matt Callanan and Alexandra Spillane [12] developed a DevOps toolchain called "Rolling Upgrade." To automate testing in Rolling Upgrade, the authors mandated (as part of the standards) that a self-testing "smoke test" script be deployed with every instance of the service—if the smoke test script was missing on a single node, the deployment (and thus, the release) would fail. As a result, the author's proposed framework reduces the deployment time by around 85%, allowing deployments to be completed in minutes instead of hours.

The paper by Pardo et al. [24] describes and implements preventive quality tools and good practices for software development that allow continuous integration and functional testing. Solano CI provides a high-performance continuous integration and deployment platform based on the author's study. Solano CI accelerates software test and deployment processes by automating and intelligently parallelizing them, which delivers 10 to 80 times faster results than current solutions.

5 CONCLUSION

Our study showed the impact of test automation practice in DevOps, testing tools at the time of automating the test cases, and the role of testing tools at different stages of testing in DevOps lifecycle. As testing automation is rapidly becoming necessary for small, medium-sized, and micro-enterprises. Automation of software testing increases software quality and efficiency. Automated test cases can be performed effectively by specific tools that can compare actual and expected results. By doing so, automated testing can ensure software proficiency without requiring repetitive manual intervention. The result section of the present study given a broader information regarding benefits of automated testing in DevOps, impact of testing tools in different phases of DevOps lifecycle. As we took google scholar as a public database for the review process, the results came out after quality assessment phase showed that not much research work had been carried out in the domain,

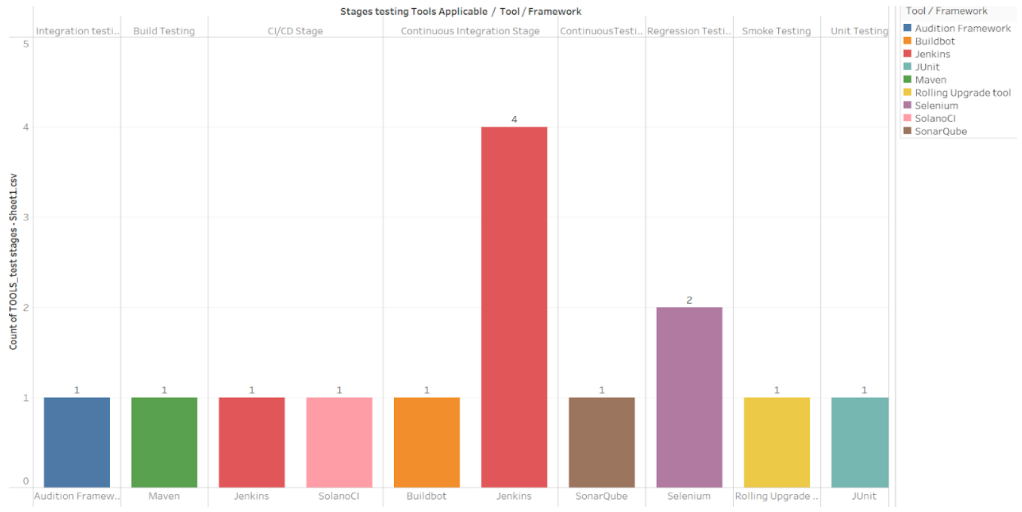

Figure 4: Above figure depicts the different Automated testing tool and its applicability in different testing stages.

i.e., test automation practice in DevOps environment. The use of limited data sources for searches is one drawback of the present study, as it limits the information regarding test strategies used in DevOps for automating the testing phase. Some of the literature deals with similarities in the methodology of different development standards, and in specific frameworks, testing stages have been included. However, this inclusion of studies does not bring large organizations that much relief when it comes to CI/CD and other stages in the DevOps environment. That's why further research is necessary, and this will only be possible when trying to find more pieces of literature and perform more research work in this field. Overall, the present study reviews the term "Test Automation as a practice in DevOps lifecycle" and analyzes the effect of automated testing in DevOps. It also explores the various automated testing tools available and their impact across multiple test stages in a DevOps environment.

REFERENCES

[1] Kim *et al.*2009. Implementing an Effective Test Automation Framework. doi: 10.1109/COMPSAC.2009.188

[2] Borgenholt, Gaute, Begnum, Kyrre, Engelstad, Paal. 2013. Audition: a DevOps-oriented service optimization and testing framework for cloud environments https://oda.oslomet.no/oda-xmlui/handle/10642/1944

[3] Roche, James.2013. Adopting DevOps practices in quality assurance Merging the art and science of software development. https://doi.org/10.1145/2538031.2540984

[4] Eran Kinsbruner 2021. The Testing Pyramid & How to Use It. https://www.perfecto.io/blog/testing-pyramid

[5] GenRocket. 2019. How to Provision Test Data for Continuous Testing. https://www.genrocket.com/newsletter/how-to-provision-test-data-for-continuous-testing/

[6] B.A. Kitchenham *et al.* 2002. Preliminary Guidelines for Empirical Research in Software Engineering. IEEE Transactions on Software Engineering, Vol. 28, No. 8, pp. 721-734, 2002.

[7] T. Greenhalgh. 2001. How to Read a Paper. 2nd Edition. BMJ Publishing Group.

[8] T. Dyba and T. Dingsoyr. 2008. Empirical Studies of Agile Software Development: A Systematic Review. Information and Software Technology.

[9] Soni, Mitesh.2015. End to end automation on cloud with build pipeline: the case for DevOps in insurance industry, continuous integration, continuous testing, and continuous delivery.

[10] Stillwell, Mark.Coutinho, Jose G. F. 2015. A DevOps approach to integration of software components in an EU research project.

[11] Kalliosaari *et al.* 2016. DevOps adoption benefits and challenges in practice: A case study

[12] Callanan *et al.*2016. DevOps: making it easy to do the right thing.

[13] Samarawickrama *et al.*2017. Continuous scrum: A framework to enhance scrum with DevOps.

[14] Setiawan *et al.*2017.Pioneering the automation of lnternal quality assurance system of higher education (IQAS-HE) using DevOps approach

[15] Agarwalet al. 2018.Continuous and integrated software development using DevOps

[16] Mohammad, Sikender Mohsienuddin.2018. Improve Software Quality through practicing DevOps Automation

[17] Akman *et al.*2018.ALM Tool Infrastructure with a Focus on DevOps Culture

[18] Oyelami *et al.*2019.Quality Assurance and Traceability in Containerized Continuous Delivery Process

[19] Cruzes *et al.*,2019. Testing in a DevOps Era: Perceptions of Testers in Norwegian Organisations

[20] Agrawal *et al.*2019.Devops, A New Approach To Cloud Development amp; Testing

[21] Faber, Frank.2020. Testing in DevOps

[22] Wang *et al.*2020.Test automation process improvement in a DevOps team: experience report

[23] Angara *et al.*2020 .Feasibility predictability model for software test automation projects in DevOps setting

[24] Sophocleous, Rafaela; Kapitsaki, Georgia M.2020.Examining the Current State of System Testing Methodologies in Quality Assurance

[25] Pardo *et al.*,2021. Documenting and implementing DevOps good practices with test automation and continuous deployment tools through software refinement

[26] Michal Doležel.2020.Defining TestOps: Collaborative Behaviors and Technology-Driven Workflows Seen as Enablers of Effective Software Testing in DevOps.